

Metody Obliczeniowe w Nauce i Technice

Laboratorium 2 — Metoda najmniejszych kwadratów

Mateusz Stoch

25 marca 2026

1 Zadanie 1 — Metoda najmniejszych kwadratów

1.1 Opis zadania

Zadanie polega na zastosowaniu metody najmniejszych kwadratów do przewidywania roku wydania utworu muzycznego na podstawie jego cech. Zbiór danych `YearPredictionMSD.txt` zawiera 515 345 utworów, z których pierwsze 463 715 stanowi zbiór treningowy, a pozostałe 51 630 to zbiór testowy. Pierwsza kolumna to rok wydania, pozostałe 90 kolumn to cechy numeryczne (barwa i kowariancja barwy).

Zadanie obejmuje:

- (a) Wczytanie zbioru i podział na część treningową i testową.
- (b) Narysowanie histogramu i posortowanego wykresu roku wydania.
- (c)–(d) Zbudowanie macierzy modelu $\mathbf{A}$ (z dodaną kolumną jedynek) oraz jej znormalizowanej wersji metodą min-max do $[0, 1]$:

$$f_{\text{scaled}} = \frac{f - f_{\min}}{f_{\max} - f_{\min}}. \quad (1)$$

- (e) Wyznaczenie wektora wag $\mathbf{w}$ przez rozwiązanie równania normalnego:

$$\mathbf{A}^T \mathbf{A} \mathbf{w} = \mathbf{A}^T \mathbf{y}. \quad (2)$$

- (f) Wyznaczenie wag metodą dekompozycji SVD (`scipy.linalg.lstsq`) oraz z regularyzacją Tichonowa ($\lambda = 0.01$):

$$(\mathbf{A}^T \mathbf{A} + \lambda \mathbf{I}) \mathbf{w} = \mathbf{A}^T \mathbf{y}. \quad (3)$$

- (g) Obliczenie współczynnika uwarunkowania macierzy $\mathbf{A}^T \mathbf{A}$ dla wersji nieznormalizowanej i znormalizowanej.
- (h) Ewaluacja modeli na zbiorze testowym przy użyciu średniego błędu bezwzględnego (MAE).
- (i) Inkrementalne wyznaczenie wag filtrem Kalmana podzielonym na 5 pakietów danych.

1.2 Argumentacja rozwiązania

1.2.1 Przygotowanie danych i normalizacja

Do macierzy cech $\mathbf{X}$ dołączono z lewej kolumnę jedynek, żeby model mógł uczyć się wyrazu wolnego (biasu). Tak powstała macierz modelu $\mathbf{A}$.

Normalizacja min-max (wzór 1) jest potrzebna, bo poszczególne cechy mogą mieć bardzo różne zakresy wartości – bez normalizacji macierz $\mathbf{A}^T \mathbf{A}$ staje się gorzej uwarunkowana i rozwiązanie traci precyzję. Analogicznie normalizuje się wektor $\mathbf{y}$. Żeby uzyskać predykcje w oryginalnej skali, stosuje się operację odwrotną:

$$\hat{y}_{\text{orig}} = \hat{y}_{\text{scaled}} \cdot (y_{\text{max}} - y_{\text{min}}) + y_{\text{min}}. \quad (4)$$

1.2.2 Równanie normalne i jego warianty

Metoda najmniejszych kwadratów minimalizuje sumę kwadratów reszt $\|\mathbf{A}\mathbf{w} - \mathbf{y}\|^2$. Rozwiązanie wyraża się przez równanie normalne (2). Zamiast jawnie odwracać macierz, używamy solwera `np.linalg.solve`, co jest numerycznie bezpieczniejsze.

Regularyzacja Tichonowa (wzór 3) dodaje do macierzy $\mathbf{A}^T \mathbf{A}$ człon $\lambda \mathbf{I}$, który poprawia uwarunkowanie i chroni przed przeuczeniem. Dla wyrazu wolnego (pierwsza kolumna jedynek) ustawiono odpowiedni element macierzy $\mathbf{I}$ na zero, żeby nie regularyzować biasu.

Biblioteczna funkcja `scipy.linalg.lstsq` rozwiązuje problem wewnętrznie przez rozkład SVD, co jest najbardziej stabilną, ale i najwolniejszą metodą.

1.2.3 Współczynnik uwarunkowania

Współczynnik uwarunkowania $\text{cond}(\mathbf{A}^T \mathbf{A})$ mówi, o ile maksymalnie mogą urosnąć błędy. Przybliżona utrata cyfr znaczących wynosi:

$$\Delta \approx \log_{10}(\text{cond}(\mathbf{A}^T \mathbf{A})). \quad (5)$$

1.2.4 Filtr Kalmana (wyznaczanie inkrementalne)

Zamiast wczytać wszystkie dane naraz, można aktualizować wagi partiami. Przy każdym pakiecie $\mathbf{A}_k$, $\mathbf{y}_k$ liczymy błąd predykcji (innowację):

$$\mathbf{e}_k = \mathbf{y}_k - \mathbf{A}_k \mathbf{w}_{k-1}, \quad (6)$$

aktualizujemy macierz informacji:

$$\mathbf{P}_k^{-1} = \mathbf{P}_{k-1}^{-1} + \mathbf{A}_k^T \mathbf{A}_k, \quad (7)$$

i poprawiamy wagi:

$$\mathbf{w}_k = \mathbf{w}_{k-1} + \mathbf{P}_k \mathbf{A}_k^T \mathbf{e}_k. \quad (8)$$

Wynik po pięciu iteracjach powinien zbiegnąć do tego, co daje rozwiązanie jednorazowe. Zamiast jawnego odwracania $\mathbf{P}$ używamy `np.linalg.solve`.

1.2.5 Kluczowe fragmenty kodu

```
1 y_train = train[0].values
2 X_train = train.drop(0, axis=1).values
3 y_test = test[0].values
4 X_test = test.drop(0, axis=1).values
5
6 ones_train = np.ones((X_train.shape[0], 1))
7 ones_test = np.ones((X_test.shape[0], 1))
8 A_train = np.hstack((ones_train, X_train))
9 A_test = np.hstack((ones_test, X_test))
10
11 X_min = X_train.min(axis=0)
12 X_max = X_train.max(axis=0)
13 X_train_scaled = (X_train - X_min) / (X_max - X_min)
14 X_test_scaled = (X_test - X_min) / (X_max - X_min)
15
16 A_train_scaled = np.hstack((ones_train, X_train_scaled))
17 A_test_scaled = np.hstack((ones_test, X_test_scaled))
18
19 y_min = y_train.min(); y_max = y_train.max()
20 y_train_scaled = (y_train - y_min) / (y_max - y_min)
```

Listing 1: Budowa macierzy i normalizacja

```
1 # (e) Równanie normalne -- solver
2 w_norm = np.linalg.solve(
3     A_train.T @ A_train, A_train.T @ y_train)
4 w_norm_s = np.linalg.solve(
5     A_train_scaled.T @ A_train_scaled,
6     A_train_scaled.T @ y_train_scaled)
7
8 # (f-1) SVD przez lstsq
9 w_svd, _, _ = scipy.linalg.lstsq(A_train, y_train)
10
11 # (f-2) Regularyzacja Tichonowa lambda=0.01
12 lam = 0.01
13 I = np.eye(A_train.shape[1]); I[0, 0] = 0
14 w_reg = np.linalg.solve(
15     A_train.T @ A_train + lam * I, A_train.T @ y_train)
```

Listing 2: Wyznaczanie wag – trzy warianty

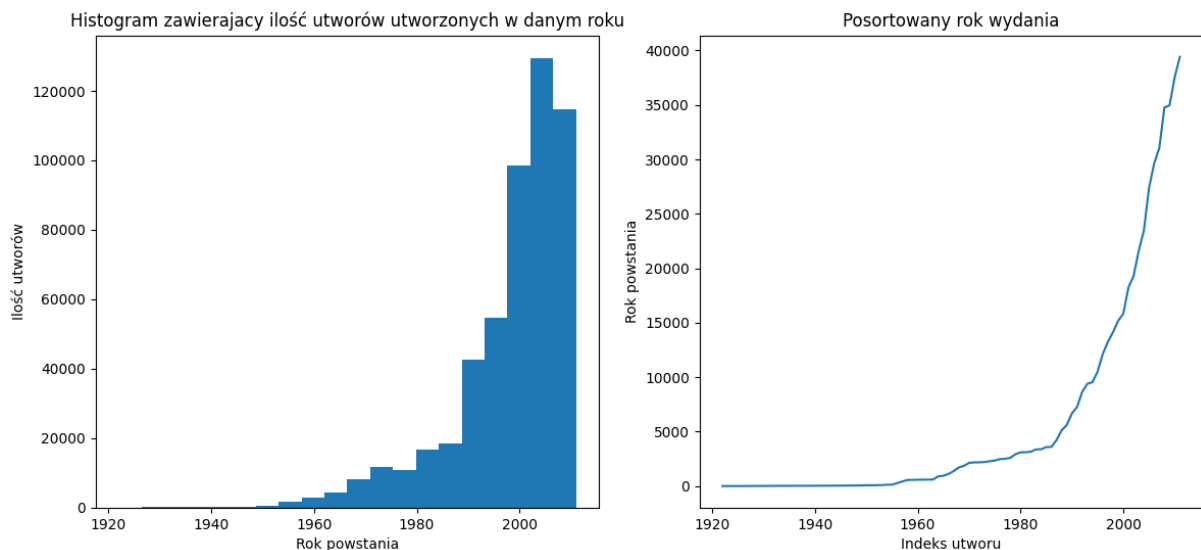
```

1 P_inv      = 1e-5 * np.eye(A_train.shape[1])
2 w_kalman   = np.zeros(A_train.shape[1])
3
4 batches    = [(0,100000),(100000,200000),(200000,300000),
5               (300000,400000),(400000,A_train.shape[0])]
6
7 for idx, (s, e) in enumerate(batches):
8     A_k = A_train[s:e]; y_k = y_train[s:e]
9     err  = y_k - A_k @ w_kalman
10    P_inv += A_k.T @ A_k
11    w_kalman += np.linalg.solve(P_inv, A_k.T @ err)
12    y_pred = A_test @ w_kalman
13    mae = np.mean(np.abs(y_test - y_pred))
14    print(f"Iteracja {idx+1}: MAE = {mae:.4f}")

```

Listing 3: Filtr Kalmana – aktualizacja partiami

1.3 Wykresy i wyniki liczbowe



Wykres 1: Histogram (lewy) i posortowany wykres roku wydania (prawy) dla całego zbioru.

Tabela 1: Współczynniki uwarunkowania macierzy $\mathbf{A}^T \mathbf{A}$.

Wersja macierzy	$\text{cond}(\mathbf{A}^T \mathbf{A})$
Nieznormalizowana	$3,87 \times 10^9$
Znormalizowana	$2,15 \times 10^6$

Tabela 2: Średni błąd bezwzględny (MAE) na zbiorze testowym dla różnych wariantów.

Metoda	MAE (lata)
Równanie normalne (nieznormalizowane)	6,800
Równanie normalne (znormalizowane)	6,800
SVD – <code>lstsq</code> (nieznormalizowane)	6,800
Regularyzacja Tichonowa $\lambda = 0.01$	6,800

Tabela 3: MAE filtra Kalmana po każdej iteracji (zbiór testowy).

Iteracja	Wiersze treningowe	MAE (lata)
1	0 – 100 000	6,846
2	100 000 – 200 000	6,807
3	200 000 – 300 000	6,800
4	300 000 – 400 000	6,801
5	400 000 – 463 715	6,800

1.4 Wnioski

1. Normalizacja min-max zmniejszyła współczynnik uwarunkowania z $3,87 \times 10^9$ do $2,15 \times 10^6$, czyli o ponad trzy rzędy wielkości (Tabela 1). Przekłada się to bezpośrednio na mniejsze ryzyko utraty cyfr znaczących podczas rozwiązywania układu.
2. Wszystkie cztery metody dały praktycznie identyczne MAE na poziomie $\approx 6,800$ lat (Tabela 2). Oznacza to, że dla tego zbioru różnice numeryczne między solverem, SVD i regularyzacją są na tyle małe, że nie wpływają na jakość predykcji.
3. Filtr Kalmana już po 3 iteracjach (300 000 wierszy) osiągnął wynik porównywalny z rozwiązaniem jednorazowym — MAE $\approx 6,800$ (Tabela 3). Podejście inkrementalne jest przydatne przy dużych zbiorach danych, które nie mieszczą się w pamięci.
4. Współczynnik uwarunkowania wpływa na liczbę traconych cyfr znaczących w mantysie zgodnie z zależnością $\Delta \approx \log_{10}(\text{cond})$. Dla nieznormalizowanej macierzy tracimy ok. 9 cyfr z 15 dostępnych w podwójnej precyzji — dlatego normalizacja jest tu kluczowa.

2 Podsumowanie

1. **Normalizacja danych.** Przed rozwiązaniem układu warto znormalizować cechy, żeby macierz $\mathbf{A}^T \mathbf{A}$ była lepiej uwarunkowana. W tym zadaniu uwarunkowanie poprawiło się o ponad trzy rzędy wielkości.
2. **Metody wyznaczania wag.** Równanie normalne z solverem, SVD i regularyzacja dają zbliżone wyniki dla dobrze przygotowanych danych. SVD jest najstabilniejszą metodą, ale też najwolniejszą. Regularyzacja przydaje się głównie przy danych, gdzie macierz może być prawie osobliwa.

3. **Podejście inkrementalne.** Filtr Kalmana umożliwia aktualizację wag bez konieczności trzymania całego zbioru w pamięci. Wynik zbiega do rozwiązania jednorazowego już po kilku iteracjach.

Bibliografia

Literatura

- [1] *Wprowadzenie do laboratorium*, materiały dostępne na platformie Teams w katalogu lab02.